18-786 Introduction to Deep Learning
Homework 2 — Training Multilayer Perceptrons

1 Scope

This assignment is on implementing backpropagation (BP) and training multilayer perceptrons (MLPs) with different optimizers.

2 Instructions

- Deadline for this assignment is February 13 via Gradescope.
- Do keep in mind the late submission policy (check the course website).
- Your submission has two parts: (i) a PDF report that details all the requested deliverables and (ii) All the code used for the assignment. You will upload the PDF report in Gradescope, marking which parts of the report correspond to which deliverable. The code will be uploaded separately, and we will run "software similarity" software on it to detect violations of the integrity policy.
- Use LaTeX to complete the PDF report, use .py files to complete the code. A starter code is provided, you should work from the given code, but you may add additional functions or python files if needed. Notebooks are not allowed.
- You can discuss this HW with others, but you are not allowed to share, borrow, copy, or look at each other's codes.
- All code that you submit must be yours (except for the starter code that we provide). Specifically, you are not allowed to use any software packages or code from the internet or other resources, except for a basic Python installation of `numpy` and `matplotlib`. Python standard library and some utility packages like `rich` and `tqdm` are also allowed.
- Reproducibility: Whenever possible, please set seeds for all randomness, so that rerunning code could produce same results, do not overwrite your code from earlier deliverables when attempting later ones.

3 Problem Set

In this homework, you will implement training and testing of MLPs with NumPy. Specifically, you will replicate several basic features of the demo at the TensorFlow Playground website:

`https://playground.tensorflow.org/`

Training and Validation Dataset

In the starter code, `numpyNN.py`, we have provided some datasets for you to use. There are five sets:

- linear-separable
- circle
- XOR
- sinusoid
- swiss-roll

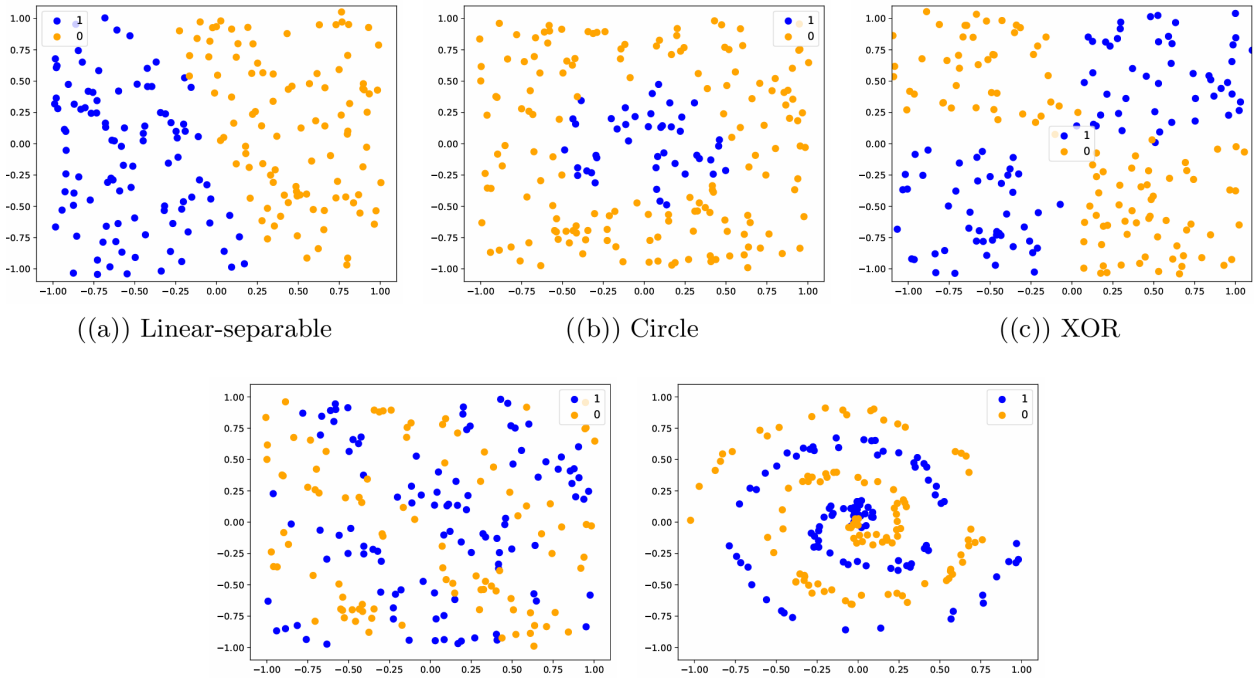
The function `sample_dataset` in the starter code will allow you to obtain samples from each. This code will give you samples from a 2-dimensional input (x_i, y_i) to a one-dimensional output d_i. The inputs (x_i, y_i) are both real-valued with $(x_i, y_i) \in [-1, +1]^2$. The output d_i is binary-valued, $d_i \in \{0, 1\}$. Figure 1 shows the five datasets you will work with.

For each model you reported in this homework, unless specified otherwise, you should also report the best validation accuracy.

Deliverable 1: Code

Your first deliverable is an implementation of a codebase that trains a generic MLP. Your implementation should provide the following capabilities:

- Ability to handle any number of layers with different number of perceptrons per layer.
- The following activation functions: ReLU, Sigmoid, tanh, and linear.
- Two possible loss functions to train with: ℓ_2^2 (L2) and cross-entropy (CE).
- Initializing MLP parameters with Xavier or He initialization (either uniform or normal variants are allowed).
- The forward pass of the MLP.
- The backward pass (a.k.a. BP).
- Using basic gradient descent to train a network; this can have constant step size or a decaying step size.
- Implementing gradient descent with momentum to train a network.
- Implementing ADAM to train a network.



((a)) Linear-separable ((b)) Circle ((c)) XOR

((d)) Sinuoid ((e)) Swiss-roll

Figure 1: Datasets: (a) Linear-separable, (b) Circle, (c) XOR, (d) Sinusoid, (e) Swiss-roll.

The entry code to this should have switches to select across various options. You are encouraged to follow good software engineering practices, but this is not mandatory. For example, you can refer to the following architecture:

```
mlp = initialize_mlp(num_layers, num_width, opt_act, opt_init)

train_mlp(mlp, training_data, num_epoch, opt_loss, opt_optim)

test_mlp(mlp, test_data, opt_loss)

def train_mlp(mlp, training_data, num_epoch, opt_loss, opt_optim):
    for epoch in range(num_epoch):
        y, z = forward(mlp, training_data.x)
        y_L = y[num_layers - 1]
        train_loss, loss_div = loss(y_L, training_data.y, opt_loss)
        dW, db = backward(mlp, training_data.x, y, z, loss_div)
        update_mlp(mlp, dW, db, opt_optim)
        ...

def test_mlp(mlp, test_data, opt_loss):
    y, _ = forward(mlp, test_data.x)
    test_loss, _ = loss(y[num_layers - 1], test_data.y, opt_loss)

    ...
```

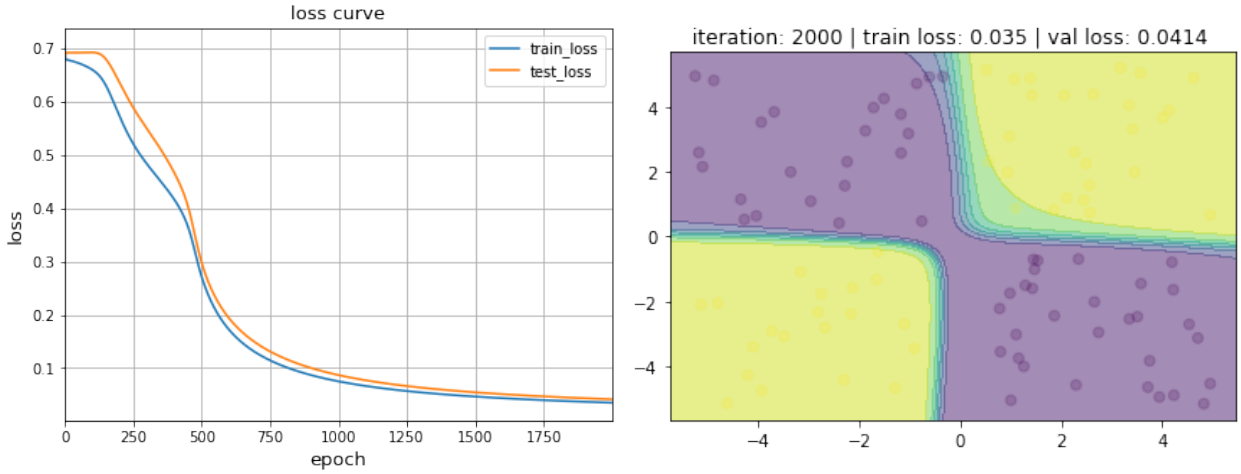


Figure 2: Example results for XOR dataset: (a) Loss curve, (b) Decision boundary.

The deliverable for this part is the code itself. Your PDF report should not contain your code. You will upload your code separately.

Deliverable 2: Linear Separable Dataset

Train an MLP on the linear-separable dataset. Sample 200 points for training and 200 points for validation. Use basic gradient descent to train an MLP with a single hidden layer that has a single perceptron under L2 loss. Choose the activation functions on each layer carefully.

Your deliverable should include:

- A plot of training/validation error with epochs.
- A visualization of the final decision boundary you obtain.

Specifically, you need to implement a codebase to produce a visualization as in Figure 2.

We expect the best validation accuracy to be at least 95%.

Deliverable 3: XOR Problem

Train an MLP on the XOR dataset. Sample 200 points for training and 200 points for validation. You have complete freedom in the design of the MLP, including the number of layers, the number of perceptrons per layer, the activations for each layer, and the loss function; however, the input has to be two-dimensional (x, y) and output is one-dimensional. You are welcome to use any of the optimizers you have implemented.

As with Deliverable 2, you should include:

- A plot of training/validation error with epochs.
- A visualization of the final decision boundary you obtain.
- Details of the architecture of the specific network used (layers, activations, cost function).

We expect the best validation accuracy to be at least 95%.

Deliverable 4: Differences in Cost Function

This deliverable looks at the difference in training a regressor vs. a classifier. Both MLPs should share the same architecture, except for final activations (which you need to select carefully) and the loss function—which is L2 for one and CE for the other. Train both MLPs with same training hyperparameters on the circle dataset, sampling 200 points for training and 200 points for validation.

Show:

- A plot of training/validation error with epochs.
- Final decision boundaries for each of the two MLPs.
- Details of the architecture of the specific networks used (layers, activations, cost function).

We expect the best validation accuracy to be at least 95% for the best performing cost function; there is no requirement for the other one.

Deliverable 5: Differences in Optimizers

This deliverable looks at the differences in training with:

- Vanilla gradient descent,
- Gradient descent with momentum,
- ADAM.

You will use the sinusoid dataset, sampling 200 points for training and 200 points for validation. Design an MLP for this dataset; you have full freedom in network parameters and loss functions, but the network must be **at least** four layers deep (3 hidden layers and an output layer). You will train it three times, once for each of the three optimization strategies.

For each of the three optimizers, show:

- A plot of training/validation error with epochs.
- Final decision boundaries.

We expect the best validation accuracy to be at least 95% for the best performing optimizer; there is no requirement for the other one.

In addition, write a short explanation of why the best-performing optimizer works well.

Deliverable 6: Swiss Roll

Train an MLP to classify on the swiss-roll dataset, sampling 200 points each for training and validation. You have complete freedom in designing the network, but it must at least be four layers deep.

In addition to detailing the specifics of the network you used, show:

- A plot of training/validation error with epochs.
- Final decision boundaries.

We expect the best validation accuracy to be at least 90%.

Deliverable 7: Non-linear Embeddings

In machine learning, embedding the input in a non-linear space often dramatically simplifies the learning problem. For example, the circles dataset becomes linearly separable if we include $x_i^2 + y_i^2$ as an input variable (do test this).

For the XOR dataset:

- Design an MLP with the fewest possible perceptrons along with a single additional non-linear function of inputs.
- Implement the code to test this.
- Show training/validation error.
- Show the decision boundary (visualized in (x, y) as before).

Repeat the above for the swiss-roll dataset. You may design and use more than one additional non-linear functions of inputs for this dataset.

We expect the best validation accuracy to be at least 93% for both datasets.